LAB 7

Task # 01

```
class Main {
  public static void main(String[] args) {
    JetFighterPlane jetfighter1 = new JetFighterPlane(
        "F-22 Raptor", 2010, 30.0, 8.0,
        2, new String[]{"Machine Gun", "Missiles", "Air-to-Air Missiles"}, 8000.0);

    CargoJetPlane cargoJet1 = new CargoJetPlane(
        "Airbus A330-200F", 2009, 25.0, 6.0,
        60.0, 10.0, 6.0);

    // Create an array of Airplane objects (polymorphism in action)
    Airplane[] airplanes = new Airplane[3];

    airplanes[0] = jetfighter1;
    airplanes[1] = cargoJet1;

    // Add another object (Jetfighterplane)
    airplanes[2] = new JetFighterPlane(
        "F-35 Lightning II", 2020, 35.0, 7.5,
        2, new String[]{"Laser Cannons", "Bombs", "Air-to-Surface Missiles"}, 10000.0);

    // Loop through the array and call the printDetails() method for each object
    for (Airplane airplane : airplanes) {
      System.out.println("\n--- Airplane Details ---");
      airplane.printDetails();
      System.out.println();
    }
  }
}
abstract class Airplane {
  // Member variables
  private String name;          // Name of the airplane
  private int yearBuilt;        // Year the airplane was built
  private double thrust;        // Thrust of the jet engine in kilonewtons (kN)
  private double fuelUsagePerKm; // Fuel usage per kilometer in liters

  // Constructor
  public Airplane(String name, int yearBuilt, double thrust, double fuelUsagePerKm) {
    this.name = name;
    this.yearBuilt = yearBuilt;
    this.thrust = thrust;
    this.fuelUsagePerKm = fuelUsagePerKm;
  }

  // Accessor (Getter) Methods
  public String getName() {
    return name;
```

LAB 7

```
    }

    public int getYearBuilt() {
        return yearBuilt;
    }

    public double getThrust() {
        return thrust;
    }

    public double getFuelUsagePerKm() {
        return fuelUsagePerKm;
    }

    // Mutator (Setter) Methods
    public void setName(String name) {
        this.name = name;
    }

    public void setYearBuilt(int yearBuilt) {
        this.yearBuilt = yearBuilt;
    }

    public void setThrust(double thrust) {
        this.thrust = thrust;
    }

    public void setFuelUsagePerKm(double fuelUsagePerKm) {
        this.fuelUsagePerKm = fuelUsagePerKm;
    }

    // Method to calculate fuel consumption for a given distance
    public double calculateFuelConsumption(double distance) {
        return distance * fuelUsagePerKm;
    }

    // Method to calculate the distance that can be covered with a given amount of fuel
    public double calculateDistance(double fuelAvailable) {
        return fuelAvailable / fuelUsagePerKm;
    }

    // Method to print the airplane details
    public void printDetails() {
        System.out.println("Airplane Name: " + name);
        System.out.println("Year Built: " + yearBuilt);
        System.out.println("Engine Thrust: " + thrust + " kN");
        System.out.println("Fuel Usage per km: " + fuelUsagePerKm + " liters/km");
    }
}
```

LAB 7

```
class JetFighterPlane extends Airplane {
   private int seatCapacity;
   private String[] weaponTypes;
   private double maxBombWeight;

   JetFighterPlane(String name, int yearBuilt, double thrust, double fuelUsagePerKm , int seatCapacity , String[]
weaponTypes, double maxBombWeight) {
      super(name, yearBuilt, thrust, fuelUsagePerKm);
      this.seatCapacity = seatCapacity;
      this.maxBombWeight = maxBombWeight;
      this.weaponTypes = weaponTypes;
   }

   int getSeatCapacity() {
      return seatCapacity;
   }
   void getWeaponTypes(){
      if (weaponTypes != null) {
         for (String weapon :  weaponTypes){
            System.out.println("-" + weapon);
         }
      }
   }
   double getMaxBombWeight(){
      return  maxBombWeight;
   }

   void setSeatCapacity(int seatCapacity) {
      this.seatCapacity = seatCapacity;
   }
   void setWeaponTypes(String[] weaponTypes){
      this.weaponTypes = weaponTypes;
   }
   void setMaxBombWeight(double maxBombWeight){
      this.maxBombWeight = maxBombWeight;
   }
   void getWeaponTypesWithMaxWeightedNuclearBomb(){
      if (weaponTypes != null) {
         for (String weapon :  weaponTypes){
            System.out.println("-" + weapon);
         }
      }
      System.out.println("The plane can travel with the nuclear Bomb of "+ maxBombWeight+ "KiloGrams");
   }
   @Override
   public void printDetails(){
      super.printDetails();
      System.out.println("Pilot Seat Type: " + seatCapacity);
      System.out.println("Weapons carried by this Jetfighter: ");
```

LAB 7

```
        getWeaponTypes();
        System.out.println("Maximum Nuclear Bomb Weight: " + maxBombWeight + " kg");
    }
}
class CargoJetPlane extends Airplane {
    // Member variable specific to CargoJetPlane
    private double length; // Length of the cargo area
    private double breadth; // Breadth of the cargo area
    private double height; // Height of the cargo area

    // Constructor
    public CargoJetPlane(String name, int yearBuilt, double thrust, double fuelUsagePerKm,
                         double length, double breadth, double height) {
        // Call the base class constructor to initialize the inherited properties
        super(name, yearBuilt, thrust, fuelUsagePerKm);
        this.length = length;
        this.breadth = breadth;
        this.height = height;
    }

    // Accessor (Getter) Methods for the new variables
    public double getLength() {
        return length;
    }

    public double getBreadth() {
        return breadth;
    }

    public double getHeight() {
        return height;
    }

    // Mutator (Setter) Methods for the new variables
    public void setLength(double length) {
        this.length = length;
    }

    public void setBreadth(double breadth) {
        this.breadth = breadth;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    // Overloaded method to calculate cargo capacity (Area)
    private double calculateCargoCapacity() {
        return length * breadth;  // Area calculation (length * breadth)
```

LAB 7

```
    }

    // Overloaded method to calculate cargo capacity (Volume)
    private double calculateCargoCapacity(double height) {
        return length * breadth * height; // Volume calculation (length * breadth * height)
    }

    // Overridden method to print the airplane details, adding more info for CargoJetPlane
    @Override
    public void printDetails() {
        // Call the base class print method first
        super.printDetails();
        // Now print CargoJetPlane-specific details
        System.out.println("Cargo Capacity (Area): " + calculateCargoCapacity() + " square meters");
        System.out.println("Cargo Capacity (Volume): " + calculateCargoCapacity(height) + " cubic meters");
    }
}
```